RutaSmart — Documentación de Firebase

Authentication y Cloud Firestore

Sarah Isabella Peña Cortes

Docente: Ing. Jairo Armando Salcedo Aranda

Universidad de San Buenaventura, Bogotá · Optativa II: Desarrollo de Aplicaciones para Dispositivos Móviles · Noviembre de 2026

Tabla de contenido

Información del documento..... 3

1. Servicios usados..... 4

2. Modelo de datos en Firestore..... 5

3. Operaciones (CRUD)..... 6

4. Sincronización..... 7

5. Reglas de seguridad..... 8

6. Manejo de errores..... 10

7. Pasos de configuración..... 11

Información del documento

Campo	Valor
Proyecto	RutaSmart — Asistente inteligente para selección de alternativas de transporte
Documento	RS-FB-001 · Documentación de Firebase
Estándar de referencia	Integración con Firebase
Versión	1.0
Autora	Sarah Isabella Peña Cortes (sipenac@academia.usbbog.edu.co)
Asignatura	Optativa II: Desarrollo de Aplicaciones para Dispositivos Móviles
Docente	Ing. Jairo Armando Salcedo Aranda
Institución	Universidad de San Buenaventura, Bogotá
Fecha	Noviembre de 2026

1. Servicios usados

Servicio	Uso
Firebase Authentication	Registro, inicio de sesión, recuperación de contraseña (correo/contraseña).
Cloud Firestore	Historial de búsquedas y preferencias por usuario; rol de administrador (admins) y estado del servicio (config/servicio).

Estado: conectado al proyecto de Firebase de la autora (app/google-services.json incluido). Falta en la consola: habilitar Correo/contraseña, crear la base de Firestore, publicar las reglas y crear el documento del administrador (Manual de Instalación §4 y §4.1). Sin google-services.json la app sigue funcionando en modo invitado.

2. Modelo de datos en Firestore

usuarios (colección)
└── {uid} (documento) ← preferencias
  ├── criterios: { tiempo, costo, comodidad, transbordos, ambiental: int,
  │ horaPico: bool, medios: [string] }
  └── consultas (subcolección)
    └── {id} (documento, UUID) ← una búsqueda
      origen: string · destino: string · fechaMillis: long
      recomendada: string · puntaje: double (0–100)
      tiempoMinutos: int · costo: int · alternativas: int · criterios: string

admins (colección, opcional) ← administradores extra, se crean A MANO en la consola
└── {uid} (documento) ← rol: "admin"

config (colección)
└── servicio (documento) ← lo publica el administrador, lo leen todos
  lineasSuspendidas: [string] · aviso: string (≤ 200) · nivel: INFO | ALERTA | URGENTE
  actualizadoMillis: long · actualizadoPor: string (uid del admin)

3. Operaciones (CRUD)

Operación	Dónde	Firestore
Crear	HistorialRepositorio.guardar → sincronizar	set usuarios/{uid}/consultas/{id}
Leer	HistorialRepositorio.descargar (al iniciar sesión)	get usuarios/{uid}/consultas
Actualizar	PreferenciasRepositorio.guardar	set usuarios/{uid} (merge)
Eliminar	HistorialRepositorio.eliminar / eliminarTodas	delete usuarios/{uid}/consultas/{id}
Leer (rol)	Administradores.esCorreoAdmin; si no, FirebaseRemoto.esAdmin	get admins/{uid}
Leer (todos)	ServicioRepositorio.actualizar (al entrar y al buscar)	get config/servicio
Actualizar (admin)	ServicioRepositorio.publicar	set config/servicio
Leer (admin)	ServicioRepositorio.resumenUso	collectionGroup(“consultas”).limit(1000)

4. Sincronización

1. Cada búsqueda se guarda primero en Room con `sincronizada = 0`.
2. `SincronizadorHistorial` recorre las pendientes con un ciclo `while`, mientras haya pendientes **y** conexión, y las sube.
3. Si una subida falla, la reintenta una vez. Si se pierde la red, se detiene y deja el resto para después.
4. Al iniciar sesión se descargan el historial y las preferencias de la nube, por si se usó otro teléfono.

5. Reglas de seguridad

```
rules_version = '2';
// Reglas de Firestore del proyecto RutaSmart (rutasmart-10006).
// Pegar TODO este archivo en Firebase > Firestore Database > Reglas > Publicar.
service cloud.firestore {
  match /databases/{database}/documents {

    // ¿Quien hace la petición es administrador?
    // Sí, si inició sesión con admin@rutasmart.com, o si existe admins/{uid} (creado a mano en la consola).
    function esAdmin() {
      return request.auth != null
        && (request.auth.token.email == 'admin@rutasmart.com'
            || exists(/databases/$(database)/documents/admins/$(request.auth.uid)));
    }

    // Lista de administradores: cada uno solo puede comprobar SU propio documento. Nadie la modifica desde la app.
    match /admins/{uid} {
      allow read: if request.auth != null && request.auth.uid == uid;
      allow write: if false;
    }

    // Estado del servicio (líneas suspendidas + aviso): lo leen todos, solo el admin lo publica.
    match /config/servicio {
      allow read: if true;
      allow write: if esAdmin()
        && request.resource.data.keys().hasOnly(['lineasSuspendidas', 'aviso', 'nivel', 'actualizadoMillis', 'actualizadoPor'])
        && request.resource.data.lineasSuspendidas is list &&
request.resource.data.lineasSuspendidas.size() <= 20
        && request.resource.data.aviso is string && request.resource.data.aviso.size() <= 200
        && request.resource.data.nivel in ['INFO', 'ALERTA', 'URGENTE']
        && request.resource.data.actualizadoPor == request.auth.uid;
    }

    // Estadísticas: el admin puede LEER (no modificar) las consultas de todos (consulta de grupo).
    match /{ruta=**}/consultas/{consultaId} {
      allow read: if esAdmin();
    }

    // Cada usuario solo puede leer y escribir SU documento (preferencias) y SU historial.
    match /usuarios/{uid} {
      allow read, write: if request.auth != null && request.auth.uid == uid;
```

```
    match /consultas/{consultaId} {
      allow read, delete: if request.auth != null && request.auth.uid == uid;
      allow create, update: if request.auth != null && request.auth.uid == uid
        && request.resource.data.origen is string && request.resource.data.origen.size() <= 80
        && request.resource.data.destino is string && request.resource.data.destino.size() <= 80
        && request.resource.data.puntaje is number
        && request.resource.data.puntaje >= 0 && request.resource.data.puntaje <= 100
        && request.resource.data.costo is int && request.resource.data.costo >= 0;
    }
   }

   // Todo lo demás está cerrado.
   match /{document=**} {
    allow read, write: if false;
   }
  }
}
```

- Cada usuario solo accede a su documento y a su subcolección.
- esAdmin() acepta el correo admin@rutasmart.com (verificado en el token de Firebase Auth) o un documento en admins.
- admins no se puede escribir desde la app; cada usuario solo puede leer **su** documento para saber si es admin.
- config/servicio es de lectura pública y solo lo escribe un admin, con campos, tipos y tamaños validados y actualizadoPor = su uid.
- La regla de grupo {ruta=**}/consultas permite al admin **leer** (no modificar) las consultas de todos para las estadísticas.
- Se validan tipos y rangos (puntaje entre 0 y 100, costo ≥ 0, textos ≤ 80 caracteres).
- Todo lo demás está cerrado.

6. Manejo de errores

FirebaseRemoto.esperar() traduce los errores:

- FirebaseAuthException → mensaje en español (ErroresAuth);
- FirebaseNetworkException → «No hay conexión con el servicio…»;
- FirebaseFirestoreException con PERMISSION_DENIED → «No tienes permiso para hacer esto…»;
- tiempo agotado (15 s) → «El servicio no respondió…».

Todos llegan como ServicioNoDisponibleException.

7. Pasos de configuración

Ver 03_Manuales/Manual_Instalacion, sección 4.